

SQL

-- Phase 0: Setup and Create Staging Table

-- (Always preserve raw data; do your work on a staging copy)

```
CREATE TABLE world_layoffs.layoffs_staging
```

```
LIKE world_layoffs.layoffs;
```

```
INSERT INTO world_layoffs.layoffs_staging
```

```
SELECT * FROM world_layoffs.layoffs;
```

-- Phase 1: Identify and Remove Duplicate Records

-- Step 1.1: Use a CTE and ROW_NUMBER() window function to track exactly identical entries

```
WITH Duplicate_CTE AS (
```

```
    SELECT *,
```

```
        ROW_NUMBER() OVER(
```

```
            PARTITION BY company, location, industry, total_laid_off, percentage_laid_off, `date`,  
stage, country, funds_raised_millions
```

```
        ) AS row_num
```

```
    FROM world_layoffs.layoffs_staging
```

```
)
```

```
SELECT *
```

```
FROM Duplicate_CTE
```

WHERE row_num > 1; -- Anything greater than 1 represents a duplicate record

-- Step 1.2: To safely delete in MySQL, create a secondary physical staging table that houses 'row_num'

```
CREATE TABLE `world_layoffs`.`layoffs_staging2` (  
  `company` text,  
  `location` text,  
  `industry` text,  
  `total_laid_off` int DEFAULT NULL,  
  `percentage_laid_off` text,  
  `date` text,  
  `stage` text,  
  `country` text,  
  `funds_raised_millions` int DEFAULT NULL,  
  `row_num` INT  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

-- Populate the second staging table

```
INSERT INTO world_layoffs.layoffs_staging2  
SELECT *,  
  ROW_NUMBER() OVER(  
    PARTITION BY company, location, industry, total_laid_off, percentage_laid_off, `date`,  
    stage, country, funds_raised_millions  
  ) AS row_num  
FROM world_layoffs.layoffs_staging;
```

-- Safely purge the duplicated lines

```
DELETE  
  
FROM world_layoffs.layoffs_staging2  
  
WHERE row_num > 1;
```

-- Phase 2: Standardizing Inconsistent Text Values

-- Step 2.1: Trim accidental blank white spaces from company labels

```
UPDATE world_layoffs.layoffs_staging2  
  
SET company = TRIM(company);
```

-- Step 2.2: Consolidate messy, overlapping naming variations (e.g., 'Crypto', 'Crypto Currency', 'CryptoBlockchain')

```
SELECT DISTINCT industry  
  
FROM world_layoffs.layoffs_staging2  
  
ORDER BY 1;
```

```
UPDATE world_layoffs.layoffs_staging2  
  
SET industry = 'Crypto'  
  
WHERE industry LIKE 'Crypto%';
```

-- Step 2.3: Remove trailing periods or punctuation from geographical fields (e.g., 'United States.')

```
SELECT DISTINCT country  
  
FROM world_layoffs.layoffs_staging2
```

```
ORDER BY 1;
```

```
UPDATE world_layoffs.layoffs_staging2  
SET country = TRIM(TRAILING '.' FROM country)  
WHERE country LIKE 'United States%';
```

-- Step 2.4: Convert text strings into native Database Date types

```
UPDATE world_layoffs.layoffs_staging2  
SET `date` = STR_TO_DATE(`date`, '%m/%d/%Y');
```

-- Modify data type configuration definition safely on the table definition

```
ALTER TABLE world_layoffs.layoffs_staging2  
MODIFY COLUMN `date` DATE;
```

-- Phase 3: Handling Missing Null & Blank Values

-- Step 3.1: Convert empty text cells (") to uniform NULL database values

```
UPDATE world_layoffs.layoffs_staging2  
SET industry = NULL  
WHERE industry = '';
```

-- Step 3.2: Perform a SELF-JOIN to fill in blank fields using matching company context rows

```
UPDATE world_layoffs.layoffs_staging2 t1
```

```
JOIN world_layoffs.layoffs_staging2 t2
  ON t1.company = t2.company
  AND t1.location = t2.location
SET t1.industry = t2.industry
WHERE t1.industry IS NULL
AND t2.industry IS NOT NULL;
```

-- Phase 4: Dropping Unusable Records & Helper Columns

-- Step 4.1: Delete records that lack both numerical data tracking parameters (completely useless for metrics calculations)

```
DELETE
FROM world_layoffs.layoffs_staging2
WHERE total_laid_off IS NULL
AND percentage_laid_off IS NULL;
```

-- Step 4.2: Drop the 'row_num' utility column we added during Phase 1

```
ALTER TABLE world_layoffs.layoffs_staging2
DROP COLUMN row_num;
```

-- Final Verification Check

```
SELECT *
FROM world_layoffs.layoffs_staging2;
```